ICTcotas design document

Oct 18, 2026 · @Francisco

Overview

ICTcotas is a standalone Pine Script v6 overlay that draws price-delivery arrays (PD arrays) from higher timeframe candles: Fair Value Gaps (FVG), Volume Imbalances (VI) and the CE (midpoint) of the longest wick in a group of wicks. It computes no signals and raises no alerts.

It works in two layers:

- **Timeframe levels**: FVGs, VIs and wick CEs of the daily, weekly and monthly candles, drawn as lines with the high, low and CE of each level, in the same style as ICTlevels.
- **FVGs by timeframe**: FVGs, and optionally wick CEs, of 15m, 1H, 4H, D, W and M candles, drawn as boxes with a CE box, a formation marker and a label.

Level	Kind code	What it is
FVG / IFVG	0	Three-candle gap. It becomes an IFVG when a candle closes through it
Volume imbalance	1	Gap between the bodies of two consecutive candles while their wicks still overlap
Wick CE	2	Midpoint of the longest wick of a group of consecutive same-side wicks
ULW CE	4	D / W / M candle with both wicks qualifying: midpoint between its upper wick CE and its lower wick CE
FVG by timeframe	3	FVG of a 15m, 1H, 4H, D, W or M candle selected in its own input group

Objects are placed with `xloc.bar_time`, and every level extends to the right margin of the chart. Levels removed by the mitigation rule disappear.

Timeframes and data sources

The script never reads candles of the chart timeframe. All levels come from `request.security` calls on higher timeframes, with `lookahead_on` and offsets `[1]`, `[2]` and `[3]`, so only completed candles are used and nothing repaints.

`htf(tf)` returns fifteen values: the current candle time `time`, and for the last three completed candles the open, high, low, close and time (the oldest one only open, high, low and close). The newest

completed candle is called 1, the one before it 2 and the one before that 3. A FVG is the triple 1-2-3 and its middle candle is 2.

Layer	Timeframes	Condition to run	Call
Daily	D	`dodaily` and chart timeframe lower than 1D (`day0k`)	One call
Weekly	W	`doweekly` and chart lower than 1W (`wk0k`)	One call
Monthly	M	`domonthly` and chart lower than 1M (`mon0k`)	One call
By timeframe	15, 60, 240, D, W	`mfEnabled(i)` for FVGs, `wkEnabled(i)` for wick CEs	One call per enabled timeframe, inside a loop

When a timeframe is disabled, its `htf` call is made with the chart timeframe instead, so the tuple always exists. The detection is simply skipped.

A new higher timeframe candle is detected when its `time` differs from the previous bar's. At that moment the previous candle has just completed and the detection functions run once for it. The first value seen on the chart is ignored in the by-timeframe loop (`mfLastT`), so history is not replayed on the first bar.

`mfEnabled(i)` combines two rules. A timeframe is on when it is ticked and `mf0k` says it is higher than the chart timeframe (never the chart's own). It is also on when `mfnext` is greater than zero and it is among the first `mfnext` listed timeframes above the chart timeframe, counted from the nearest. `wkEnabled(i)` enables wick CEs only for 1H and 4H, through their own checkboxes, with the same `mf0k` rule. `processTf` skips the kind 0 FVG / IFVG detection of D, W or M when `mfEnabled(tfi + 3)` is on, because the same gaps are then drawn as kind 3 boxes; VIs and wicks of that timeframe are still detected.

Inputs

Inputs are grouped by type of level. Related controls share a row with `inline`. Strings are turned into constants by `getstyle()` and `getsize()`.

Group	Inputs and effect
General	Daily, weekly and monthly on or off; extra line width per timeframe step (`wextra`, weekly gets one step more than daily and monthly two); mitigation rule (`mitig`): Never, CE touched, Fully filled (wick), Fully filled (close)
Colors (by timeframe)	Bull and bear colors for daily, weekly and monthly, shared by every PD array of that timeframe
Labels	Show labels; bold; direction (+/-); size; offset in bars; label text per timeframe (D, W, M) and per part (hi, lo, CE)

Group	Inputs and effect
Fair Value Gaps	Draw, keep (up to 20), minimum size; line style, width and CE style; show hi, lo, CE; no fill and transparency; label; IFVG inversion and its label; overlap rule (Most recent, Enclosing, Keep all); extend the range to include VIs with candle 2
Volume Imbalances	Draw, keep, minimum size; styles; show hi, lo, CE; no fill and transparency; label
Wick CE	Draw, keep, minimum wick as a percentage of the candle range (default 30); line style and width; upper and lower wick labels; ULW CE (`doulw`) and its label (`txtulw`)
FVGs and wick CE by timeframe	Draw; plus N next higher timeframes; 15m, 1H, 4H, D, W, M tick boxes; keep per timeframe and minimum size; bull and bear colors per timeframe; outline style and width; fill (`mffill`: never, at margin (default), always) and transparency; formation marker; CE box and its style; labels with TF, FVG/IFVG and direction parts; label size, position and vertical anchor; wick CE for 1H and 4H; right margin layout (`mfmargin`, off) with offset (`mfmoff`, 4) and box width (`mfmw`, 4); extend levels (`mfext`, off) to the point set by `mfextto` (“for” `mfextn` (20) candles, “to start” of the FVG, or “to creation”, the end of its third candle)

The by-timeframe group has several defaults chosen on purpose. It is on by default (1H and 4H ticked), the fill is off, the CE box has zero height and the label position is Inside right. The label anchor is the CE of the FVG unless set to High or Low.

Data structures

There is a single user-defined type, `Lvl`. One instance is one level, and it also holds the handles of its chart objects, so killing the level means deleting those objects.

Field	Meaning
`kind`	0 FVG or IFVG, 1 volume imbalance, 2 wick CE, 3 FVG by timeframe, 4 ULW CE
`tfi`	Timeframe index: 0 daily, 1 weekly, 2 monthly, and 3 to 8 for 15m, 1H, 4H, D, W, M of the by-timeframe layer
`dir`	1 bullish (support), -1 bearish (resistance). Lower wicks count as bullish, upper wicks as bearish
`hi`, `lo`, `ce`	Upper edge, lower edge and midpoint
`t`	Left edge time. Never earlier than `tfirst`, the first bar of the chart
`wsize`	Size of the wick, used to pick the longest wick of a group

Field	Meaning
`inv`	True once an FVG has been inverted into an IFVG
`dead`	The level was removed and awaits cleanup
`tfm`	Formation time of an FVG: the time of the bar on which its last candle completed
`hiln`, `loln`, `celn`	Lines at the high, low and CE
`hilb`, `lolb`, `celb`	Labels of those lines (for kind 3, `celb` holds the single label)
`bx`, `cebx`	Fill rectangle, and for kind 3 the outline box and the zero-height CE box
`vln`	Vertical formation marker of a kind 3 level

Global state:

Variable	Content
`lvls`	`array<Lvl>` with every live level of all kinds and timeframes
`wcur`	`array<Lvl>` of 12 entries, indexed `tfi * 2 + side` (side 0 upper wick, 1 lower wick). It holds the level of the wick group currently open for each timeframe and side
`mfSecs`, `mfTfs`, `mfNames`	Seconds, `request.security` strings and display names of the six by-timeframe candidates
`mfSel`	Which of them are ticked
`mfLastT`	Last candle time seen for each, used to detect a new candle
`tfirst`	Time of the first chart bar
`note`	Bottom-right table with the “use a lower timeframe” warning

Detection logic

Detection runs only when a higher timeframe candle completes. The candle triple is named 1 (newest), 2 (middle) and 3 (oldest). Everything is computed from the open, high, low and close of those candles.

Fair Value Gaps

`addFvgs` finds at most one bullish and one bearish FVG per call.

- Bullish: the low of candle 1 is above the high of candle 3. The range is high of 3 to low of 1.
- Bearish: the high of candle 1 is below the low of candle 3. The range is high of 1 to low of 3.
- The gap must be at least `minsz` points (`fvgmin` for kinds 0, `mfmin` for kind 3).
- The left edge is the time of the middle candle, and the formation time `tfm` is the time at which the third candle completed.

Extending the range with volume imbalances

With `fvgvi` on, a bullish FVG is widened when candle 1 or candle 3 forms a volume imbalance with candle 2. The two cases are the gap between the body tops and bottoms of candle 2 and candle 3, and between candle 1 and candle 2. Any body gap of at least `vimin` counts, whether the wicks of the two candles overlap (a VI) or not (a gap, as when candle 1 opens above candle 2's high), the same rule FVGS+IFVG+OTE uses for its suspension blocks. The FVG then takes the lower of the two lows and the higher of the two highs. Bearish FVGs are the mirror image. The VIs are still drawn as VIs on their own (that layer keeps the strict rule, wicks overlapping).

Volume imbalances

`processTf` tests candle 1 against candle 2. A bullish VI exists when the bottom of candle 1's body is above the top of candle 2's body while the low of candle 1 is not above the high of candle 2. A bearish VI is the mirror. The range is the gap between the two bodies.

Wick groups

`addWicks` classifies the upper and lower wick of the completed candle. A wick counts when it is at least `wkmin` percent of the candle range. `wickStep` then runs a small state machine per timeframe and side:

State of the group	Wick qualifies	Wick does not qualify
No open group	Start a group with a new level at this wick	Nothing
Open group, wick longer than the stored one	Move the level to this wick: new range, CE and time	The group ends
Open group, wick not longer	Keep the stored level	The group ends
Open group whose level was removed and wick longer	Start a new level	The group ends

A group is a run of consecutive candles with a qualifying wick on the same side, so the level always shows the CE of the longest wick of the run. A lone qualifying wick forms its own group.

Overlap rule

`fvgOk` runs before an FVG of kind 0 or 3 is stored. For FVGs of the same kind, timeframe and direction that are not inverted:

- **Keep all**: nothing is filtered.
- **Most recent**: older FVGs that overlap the new one are killed.
- **Enclosing**: the new FVG is dropped if an existing one already contains it. Otherwise older overlapping FVGs are killed as with Most recent, so if none includes all of them, the newest wins.

Capacity

After a level is added, `addLvl` counts the live levels of the same kind and timeframe. If the count is above the keep limit, the oldest one is killed. The limit is `nfvg`, `nvi`, `nwk` or `nmf` for kinds 0, 1, 2 and 3.

Inversion and mitigation

`runMit` runs on every chart bar. With `doifvg` on, a live FVG (kind 0 or 3) is inverted when a candle of its own timeframe closes beyond its far edge, never on a chart close. The per-bar array `tfNew` (9 entries, by `tfi`) is set when a candle of that timeframe completed on this chart bar (`newD` / `newW` / `newM`, or a new `t0` in the by-timeframe loop), and the `var` array `tfClose` holds that candle's close (`dC1`, `wC1`, `mC1`, `u1`); `runMit` tests only levels whose `tfi` has `tfNew`, against `tfClose`. NinjaTrader: `CloseTf(tfi, close)` from `OnHtfBar` with `Closes[k][off]` does the inversion and the Fully filled (close) rule (levels flipped there skip the next `RunMit`, `flippedNow`); `RunMit` keeps only the CE and wick rules. Tradovate: `applyBar` passes `tfc` (tfi to the completed candle's close) to `runMit`, for the inversion and the close rule. ICTScalp and its generated variants copy this. Beyond the far edge means below the low of a bullish one, above the high of a bearish one. It becomes an IFVG, its direction flips, and it is renamed. After the flip the usual mitigation rule applies to it. All other levels, and FVGs when `doifvg` is off, use the mitigation rule chosen in General:

Rule	A bullish level is removed when	A bearish level is removed when
Never	Never	Never
CE touched	`low <= ce`	`high >= ce`
Fully filled (wick)	`low <= lo`	`high >= hi`
Fully filled (close)	`tfNew` and `tfClose < lo` (a close of the level's own timeframe)	`tfNew` and `tfClose > hi`

Execution pipeline

The script does little work on most bars. Objects are only created and moved on the last bar.

[embedded content: execution pipeline · detection per candle, mitigation per bar]

Order in the source. The three `htf` calls for daily, weekly and monthly run first, then `newD`, `newW` and `newM` call `processTf` for each timeframe whose candle just changed. The loop over the six by-timeframe candidates comes next. It calls `htf` again for each enabled timeframe and runs `addFvgs` and `addWicks` when its candle time differs from the last one stored. `runMit` closes the detection part of every bar.

Formation time. For levels of kinds 0 to 2 the formation time is the chart bar on which the new candle was detected. For kind 3 it is the candle time returned by the call, which is the start of the candle that has just opened.

Last bar. On `barstate.islast` every level in `lvls` is drawn: kind 3 through `drawM`, the others through `draw`. The right edge is `tright(xoff)`, so all objects follow the last bar. Objects already created are updated rather than recreated.

Warning. When none of daily, weekly and monthly is available at the chart timeframe, a table in the bottom-right says to use a chart lower than 1M.

Drawing

All objects use `xloc.bar_time`. The left edge is the level's time `t` and the right edge is `tright(xoff)`, a few bars past the last bar. Colors come from `tcol`, which maps the timeframe index and direction to the input colors.

Timeframe levels (kinds 0 to 2)

`draw` creates or updates, in this order:

1. A filled rectangle between `hi` and `lo`, only when `kfill` is true. Its transparency is `fvgtr` for FVGs and `vitr` for the rest.
2. A high line, a low line and a CE line, each only when `kshow` allows it. Wick levels show only the CE.
3. A label at the right end of each line when labels are on, or a single label at the CE (text without the part, "D VI") when the labels would overlap: the distance between neighbouring labels (half the range when the CE is shown, else the range) is below `lblmrg` times the daily ATR `dAtr`. `dAtr` is an RMA(14) of the true range of completed daily candles (chart candles on D charts and above), updated when a new daily candle starts; the ports compute it the same way from their daily candles. Labels are not created by `draw` itself: it queues them (`LblE`: level, part, price, text, color, rank) in `lbls`, and `lblApply` sorts them by price, groups those within `lblmrg * dAtr` of the first of a group, and draws one label per group with the texts joined by " / ", at the price and color of the entry with the lowest rank (the lowest price on ties). Label slots a level did not get this time are deleted.

Attenuation (`attLvl`, 2 by default) uses the ICTlevels functions `attE`, `attL`, `attW` and `attT` with the rank of the level (`rank`): weekly and monthly levels are important, daily and 4H normal, 15m and 1H minor, and VIs, wick CEs and ULW CEs one rank below FVGs of their timeframe. Line and label colors, widths and fill transparencies of all three drawing methods go through them. The rectangles of kinds 0 and 1 use `fillT` instead: `attT` up to level 1, a faster fade at level 2 (0.35 per dimming step, capped at 0.9) and no fill at levels 3 and 4, except for VIs no wider than 0.1 `dAtr` while there are at most three of them (`vinar`, counted on the last bar). In the right margin layout (`drawMM`) the boxes, labels and links to the boxes use rank 0; only the extended levels over the candles use the level's rank. Tradovate blends faded colors into `attbg` because it ignores color alpha.

Line width is the base width of the kind plus `tfi * wextra`, so weekly levels are thicker than daily ones and monthly ones thicker still. High and low lines use the hi/lo style of the kind, the CE line its CE style. Wick levels use their single style for the line.

Label text is built by `lbltxt` as the timeframe text, the level name, the optional direction sign and the part (hi, lo or CE), for example "D FVG+ hi" or "W UW CE". The level name is the FVG or IFVG text for kind 0, the VI text for kind 1, and the upper or lower wick text for kind 2.

FVGs by timeframe (kind 3)

`drawM` draws a box with the level color as its border and, only when `mffill` is "always", a transparent fill. `drawMM` fills its margin box unless `mffill` is "never". Then it adds:

- **CE box**: a box whose top and bottom are both the CE, so it has zero height and shows as a line. Its style is set by the CE outline input.

- **Formation marker**: a vertical line from the high to the low at the formation time `tfm`, drawn only when `tfm` is not before the left edge.
- **Label**: text made of the timeframe name, the FVG or IFVG name and the direction sign, each optional. Its horizontal position is the box left, center or right, or the right margin. Its vertical anchor is the high, the CE or the low. The label style is chosen to match the position, for example a right-aligned label for Inside right.

Right margin. With `mfmargin` on, `drawMM` is used instead of `drawM`. The column of a timeframe is its rank among the enabled timeframes (`mfCol`), so the box spans from bar `bar_index + mfmoff + col * (mfmw + 1)` to `mfmw` bars after it, at the FVG high and low. Only the margin boxes, CE boxes and labels use `xloc.bar_index` (a few hundred bars at most to the right of the last bar): bar times to the right of the last bar collapse to one point while the market is closed (weekends), which turned the boxes into vertical lines. Everything else keeps bar time. It gets the CE box and a label centered in the box at the chosen anchor, on two lines so the boxes can be thin: the optional timeframe (with an “i” appended for an IFVG, “1Hi”), then an up (bullish) or down (bearish) arrow from the current direction. With `mfext` on, the high, low and (with the CE box) the CE are drawn by bar time as lines in the `hiln`, `loln` and `celn` fields, from the start chosen by `mfextto` to the last bar: the time of the bar `mfextn` bars before the last one (“for”), the FVG start `t` (“to start”) or its formation time `tfm`, the end of the third candle (“to creation”), and continued by short bar-index links (`hiln2`, `loln2`, `celn2`, `mkLineI`) from the last bar to the left edge of the box. Bar time is kept for the long part because a bar index far in the past can raise a runtime error. No formation marker is drawn.

Updating

Because lines, labels and boxes are created once and updated afterwards (`mkLine`, `mkLbl` and the setters in `draw` and `drawM`), the number of objects stays bounded by the keep limits. Each kind and timeframe holds at most its `keep` count, and the chart limits are set to 500 for lines, labels and boxes.

Function reference

Every function that would otherwise return nothing ends in `true`, because Pine needs a return value to compile.

Inputs and setup helpers

Function	Behavior
`getstyle(s)`, `getsize(s)`	Turn input strings into line styles and label sizes
`mfOk(s)`	Whether a timeframe of `s` seconds is usable: strictly higher than the chart timeframe
`mfEnabled(i)`	Whether the FVGs of by-timeframe candidate `i` are drawn: ticked, or among the next `mfnext` higher ones, and always above the chart timeframe
`wkEnabled(i)`	Whether wick CEs are computed for 1H or 4H candidates
`mfcol(i, d)`, `tcol(tfi, d)`	Color of a by-timeframe candidate or of any timeframe index, by direction

Function	Behavior
`ksty`, `kcesty`, `kwid`, `kshow`, `kfill`	Style, CE style, width, visibility of a part and fill flag for a kind of level
`lbltxt(l, part)`	Label text of a level and part

Object helpers

Function	Behavior
`rmLine`, `rmLabel`, `rmBox`	Delete an object if it exists
`tright(n)`	Time `n` bars to the right of the current bar
`mkLine`, `mkLbl`	Create or update a horizontal line or a label
`kill(l)`	Marks a level dead and deletes all its objects
`draw(l)`	Draws a level of kind 0, 1 or 2
`drawM(l)`	Draws a level of kind 3
`mfCol(i)`, `drawMM(l)`	Margin column of a timeframe; draws a level of kind 3 in the right margin

Detection and maintenance

Function	Behavior
`htf(tf)`	`request.security` for the 15 values of the last three completed candles
`processTf(tfi, ...)`	Runs the FVG, VI and wick detection for one completed daily, weekly or monthly candle
`addFvgs(kind, tfi, minsz, ...)`	Finds bullish and bearish FVGs of a candle triple, applies the VI extension and stores them
`addWicks(tfi, ...)`	Classifies the wicks of a candle and feeds both sides to `wickStep`
`wickStep(tfi, side, q, sz, lo, hi, t)`	Wick group state machine
`addLvl(kind, tfi, dir, lo, hi, t)`	Creates a level, applies the overlap rule for FVGs and enforces the keep limit. Returns `na` for a discarded FVG
`fvgOk(kind, tfi, dir, lo, hi)`	Applies the overlap rule. Returns false when the new FVG is dropped
`runMit()`	Inverts FVGs into IFVGs (on a close of their own timeframe) and removes mitigated levels, on every bar

Limitations and extension points

Limitations

- The code has not been compiled or run in TradingView. Expect to fix small errors on first load.

- Levels exist only for candles completed after the script's first chart bar, and `request.security` history is limited by the chart's bar count, so deep history is not reproduced.
- Only timeframes strictly higher than the chart are used, never the chart's own: daily levels need a chart lower than 1D, weekly lower than 1W, monthly lower than 1M, and the by-timeframe layer uses only timeframes above the chart (`mfOk`).
- Detection uses the exchange candles of the symbol, so weekly and monthly boundaries follow the exchange session, not the platform timezone.
- A wick group is evaluated candle by candle. If a level is removed by mitigation while its group is still open, a longer wick later in the same group starts a new level.
- Many enabled timeframes mean many `request.security` calls. One call is made per enabled by-timeframe candidate on top of the three fixed ones.
- All levels share the 500 line, label and box limits. The keep inputs bound the count per kind and timeframe, not in total, so enabling every kind on every timeframe can exceed the limits and TradingView will delete the oldest objects.

Extension points

- A new kind of level: add a kind code, a detection function that calls `addLvl`, entries in `ksty`, `kcesty`, `kwid`, `kshow`, `kfill` and `lbltxt`, and a keep limit in `addLvl`.
- A new by-timeframe candidate: extend `mfSecs`, `mfTfs`, `mfNames`, `mfSel`, `mfLastT`, `mfcol`, the loop bound and `wcur`, and add its inputs.
- A new mitigation rule: add an option to `mitig`, a code in `mmode`, and a branch in `runMit`.

ULW CE (kind 4)

`addWicks` computes `uq` / `lq` (upper / lower wick at least `wkmin` % of the range) for each completed candle. With `doulw` on and `tfi < 3` (D, W, M only, never the 1H / 4H wicks), when both are true it adds `addLvl(4, tfi, dir, p, p, t1)` with `p = ((topC + h1) / 2 + (l1 + botC) / 2) / 2`, the midpoint of the two wick CEs, and `dir` 1 when the candle closed at or above `p`, else -1. It is drawn like a wick level (CE only, `wksty` / `wkwid`, label `txtulw`), is kept with `nwk` per timeframe and is removed by the general mitigation rule. NinjaTrader: `DrawUlw` / `UlwText` in `AddWicks`; Tradovate: `doUlw` / `txtUlw` in `addWicks`.

Bad data filter

Data errors can leave a candle with a huge wick (a bad print thousands of points away). `badatr` (6 by default, 0 off) treats a wick longer than `badatr` times `dAtr` as one: `clH` / `clL` return the body top / bottom instead of the high / low. They clean the chart candle (`cH` / `cL`, used by mitigation and the turtle soup trackers), the three candles of each D / W / M request on a new candle (before `processTf`) and those of the FVG / wick requests (before `addFvgs` / `addWicks`). `dAtr` itself is updated from the daily candle cleaned with the ATR so far, so a bad print does not inflate it. No request is added. NinjaTrader reads every high and low through `CH` / `CL` (property `BadAtr`); Tradovate cleans each chart bar at the start of `applyBar` (`badAtr`), so its candles built from chart bars leave the bad print out (their extreme is that of the other chart bars rather than the body).

Turtle soup / continuation

Five `TSt` trackers (15m, 1H, 4H, D, W) keep the previous completed candle (`po`, `ph`, `pl`, `pc`) and the forming one (`o`, `h`, `l`), built from chart bars: `tsStep` starts a candle when `timeframe.change` of its timeframe fires (the previous close is the previous chart bar's close) and extends it otherwise. `tsCont` is the continuation test (previous down: the forming candle traded above its open and price is below the previous close; mirrored). `armed` records that a continuation was seen and `inv` that a chart bar then closed back beyond the previous close, which cancels it for the rest of the candle. For the chart timeframe the forming candle is the forming chart bar, whose closes say nothing about what happened inside it, so `varip` state (`tsTkBar`, `tsTkArm`, `tsTkInv`, not rolled back between realtime ticks) tracks armed / cancelled tick by tick and is or-ed into its `inv` (NinjaTrader with OnEachTick / OnPriceChange and Tradovate keep the same tick state outside the rolled-back state). `tsSig` returns 1 / -1 for a turtle soup up / down (previous low taken and price above the previous close, or the mirror), 2 / -2 for a continuation, 0 otherwise. On the last bar a table (`tsTbl`, position `tspos`) shows one triangle per timeframe: 15m-4H up to 15m charts, 1H-D up to 1H, then 4H, D and W while not below the chart timeframe. NinjaTrader recomputes the forming candle by scanning the chart bars after the last completed bar of the 15m / 60m / 240m / daily / weekly series and draws one TextFixed per color padded with no-break spaces; Tradovate keeps `armed` / `inv` in the forming candle of its HTF aggregators.

SET check (Daily Settings)

Inputs `setmode` (auto / on / off, default auto) and `setlbl` (group `G_DAY`). `setOn` requires `syminfo.type == "futures"` and `dayOk`. Two extra daily requests, `htfT(ticker.modify(syminfo.tickerid, settlement_as_close = settlement_as_close.on), "D")` and the same with `.off`, return the same candle triples as `htf("D")` (they ask for the chart timeframe when `setOn` is off). On `newD`, `setScore()` of each version is the largest of the volume imbalance between candles 2 and 1 (bodies, wicks overlapping, as in `processTf`) and the upper and lower wick of candle 1; the settlement version with `setmode` on, the other with off, and with auto the version with the larger score (ties: without settlement) overwrites `dO1` to `dC3` before `processTf(0, ...)` and `tfClose`, and `setUsed` holds "SET on" / "SET off" for the `top_right` table drawn on the last bar when `setlbl` is on. Not ported: NinjaTrader daily bars come from the data provider, and Tradovate builds its daily candles from the chart bars.